

```
import java.util.*;
import java.util.stream.Collectors;

class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

public class Main {
    public static void main(String[] args) {
        List<Person> people = Arrays.asList(
            new Person("John", 25),
            new Person("Jane", 17),
            new Person("Alex", 20),
            new Person("Emma", 18)
        );
    }
}
```

```

);

// Filter people older than 18 and collect their names
List<String> names = people.stream()
    .filter(person -> person.getAge() > 18)
    .map(Person::getName)
    .collect(Collectors.toList());

// Print the list of names
System.out.println(names);
}
}

```

Output

```

^ java -cp /tmp/sfmFoUAMJq/Main
  [John, Alex]

=== Code Execution Successful ===

```

```
import java.util.*;
```

```
// Product class
```

```
class Product {
    private String id;
    private String name;
    private double price;
    private int stock;

```

```
public Product(String id, String name, double price, int stock) {  
    this.id = id;  
    this.name = name;  
    this.price = price;  
    this.stock = stock;  
}
```

```
public String getId() {  
    return id;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public double getPrice() {  
    return price;  
}
```

```
public int getStock() {  
    return stock;  
}
```

```
public void reduceStock(int quantity) {  
    if (stock >= quantity) {  
        stock -= quantity;  
    } else {  
        throw new IllegalArgumentException("Insufficient stock for product: " + name);  
    }  
}
```

```
    }  
  }  
}
```

// Order class

```
class Order {  
    private Map<Product, Integer> productList = new HashMap<>();  
  
    public void addProduct(Product product, int quantity) {  
        if (product.getStock() >= quantity) {  
            product.reduceStock(quantity); // reduce stock when adding product to order  
            productList.put(product, quantity);  
        } else {  
            throw new IllegalArgumentException("Insufficient stock for product: " + product.getName());  
        }  
    }  
  
    public double getTotalAmount() {  
        return productList.entrySet().stream()  
            .mapToDouble(entry -> entry.getKey().getPrice() * entry.getValue())  
            .sum();  
    }  
  
    public void displayOrderDetails() {  
        productList.forEach((product, quantity) -> {  
            System.out.println("Product: " + product.getName() + ", Quantity: " + quantity + ", Price: " +  
product.getPrice());  
        });  
        System.out.println("Total Amount: " + getTotalAmount());  
    }  
}
```

```
}  
}  
  
// Catalog class  
class Catalog {  
    private List<Product> products = new ArrayList<>();  
  
    public void addProduct(Product product) {  
        products.add(product);  
    }  
  
    public Product findProductById(String id) {  
        return products.stream()  
            .filter(product -> product.getId().equals(id))  
            .findFirst()  
            .orElse(null);  
    }  
}  
  
// OrderManager class  
class OrderManager {  
    private Catalog catalog = new Catalog();  
  
    public void addProductToCatalog(Product product) {  
        catalog.addProduct(product);  
    }  
  
    public void placeOrder(Order order) {  
        order.displayOrderDetails();  
    }  
}
```

```
}

public Product getProductById(String id) {
    return catalog.findProductById(id);
}
}

// Main class to run the program
public class Main {
    public static void main(String[] args) {
        OrderManager orderManager = new OrderManager();

        // Add products to the catalog
        Product product1 = new Product("P001", "Laptop", 1000, 5);
        Product product2 = new Product("P002", "Phone", 500, 10);
        orderManager.addProductToCatalog(product1);
        orderManager.addProductToCatalog(product2);

        // Place an order
        Order order = new Order();
        order.addProduct(product1, 2); // Add 2 Laptops to the order
        order.addProduct(product2, 3); // Add 3 Phones to the order

        // Display order details
        orderManager.placeOrder(order);
    }
}
```

Output

```
java -cp /tmp/M0AeS1jLv7/Main  
Product: Laptop, Quantity: 2, Price: 1000.0  
Product: Phone, Quantity: 3, Price: 500.0  
Total Amount: 3500.0
```

```
=== Code Execution Successful ===
```

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String paragraph = "Java is fun, Java is powerful. Java is platform-independent.";
```

```
        // Split the paragraph into words, considering punctuation and case sensitivity
```

```
        String[] words = paragraph.toLowerCase().replaceAll("[^a-z ]", "").split("\\s+");
```

```
        // Store the frequency of each word in a HashMap
```

```
        Map<String, Integer> wordCount = new HashMap<>();
```

```
        for (String word : words) {
```

```
            wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
```

```
        }
```

```
        // Display each word and its frequency
```

```
        wordCount.forEach((k, v) -> System.out.println(k + ": " + v));
```

```
}  
}
```

Output

```
java -cp /tmp/KqnkMEk22d/Main  
java: 3  
powerful: 1  
is: 3  
platformindependent: 1  
fun: 1  
  
=== Code Execution Successful ===
```

```
import java.util.*;
```

```
import java.util.stream.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        List<Integer> scores = Arrays.asList(70, 85, 90, 60, 75, 80, 95);
```

```
        // Calculate average score
```

```
        double average = scores.stream()
```

```
            .mapToInt(Integer::intValue)
```

```
            .average()
```

```
            .orElse(0.0);
```

```
        // Find maximum score
```

```
        int maxScore = scores.stream()
```

```
            .mapToInt(Integer::intValue)
```



```
.max()

.orElse(0);

// Count how many students scored above the average
long countAboveAverage = scores.stream()
    .filter(score -> score > average)
    .count();

// Print the results
System.out.println("Average Score: " + average);
System.out.println("Maximum Score: " + maxScore);
System.out.println("Count of students above average: " + countAboveAverage);
}
}
```

Output

```
java -cp /tmp/tLs5H9MuxJ/Main
Average Score: 79.28571428571429
Maximum Score: 95
Count of students above average: 4

=== Code Execution Successful ===
```